

Project 02

Laboratórios de Sistemas e Serviços

Mário Antunes

2026

Projects

Form groups of two or three students (exceptionally, projects can be done individually) and select **one** of the following projects. Due to all underlying issues with GitHub and GitHub Classroom; all projects will be submitted via **eLearning** in a single compressed file (ZIP), following a standard repository structure.

The ZIP file must contain all relevant scripts, configuration files, a `docker-compose.yml`, and a `README.md` with instructions on how to deploy the project. It should also contain a project report in PDF format.

This is a four-week project (deadline 18/06/2026). You have until the end of this week to notify your professor (via e-mail) of your group members and chosen topic (the list of topics can be found [here](#)).

Do not forget to contact your professor with any questions. Further instructions may be added.

Topics

1. IoT Environmental Monitoring System

- **Description:** Simulate multiple IoT sensors (temperature, humidity, air quality) publishing data via **MQTT** to a central broker. A collector service must store this data in a relational database. Use **Grafana** (or a custom frontend) to visualize the historical trends and current status of each sensor. The entire stack must be orchestrated with Docker Compose.
- **Core Topics:** Docker Compose, MQTT (Mosquitto), SQL Persistence, Data Visualization.

2. Real-time Collaborative Task Board

- **Description:** Build a Kanban-style task management application where multiple users can add, move, and edit tasks in real-time. Use **WebSockets** to synchronize the state across all connected clients instantly. All changes must be persisted in a database (SQL or NoSQL) to ensure data is not lost when containers are restarted.
- **Core Topics:** WebSockets, Database Persistence, Frontend Synchronization, Docker Compose.

3. Personal Finance Analytics Platform

- **Description:** Create a RESTful API for tracking personal expenses and income. The system should support categorization and multiple accounts. The frontend must feature interactive charts (e.g., using **Chart.js** or **D3.js**) to visualize spending patterns, monthly budgets, and financial health.
- **Core Topics:** FastAPI/REST, Relational Database (MariaDB/Postgres), Charting/Visualization, Docker Compose.

4. Media Library with Automated Metadata

- **Description:** Build a digital library manager for movies or books. When a user adds a title, the backend fetches rich metadata (posters, ratings, synopses) from an **external REST API** (e.g., TMDb or Google Books). Store the library in a database and provide a visual gallery with filters.
- **Core Topics:** External API Integration, SQL, Multimedia Metadata, Docker.

5. Real-time Online Auction Platform

- **Description:** Implement a bidding system where users can participate in live auctions. Use **WebSockets** to broadcast new bids to all participants immediately. The system must handle bid concurrency using a message broker (like Redis) and persist the final results in a database.
- **Core Topics:** WebSockets, Redis (Pub/Sub), SQL, Atomic Transactions.

6. Geo-Data Dashboard (Traffic or Weather)

- **Description:** Build a web dashboard that visualizes geographical data. You must create a **Python** script that uses an API to get Weather or traffic data or (using **Pandas** or **Polars**) that processes

a dataset (e.g., a CSV of weather stations or traffic incidents with Lat/Lon coordinates) and exports it to JSON. Then, deploy a **Web Server** container (Nginx or Apache) hosting an HTML page. This page must use the **Leaflet** JavaScript library to read that JSON data and display markers on an interactive map.

- **Core Topics:** Web Programming (HTML/JS/Leaflet), Data Formatting (CSV to JSON), Docker, Web Servers.